SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of india)
Re-Accredited by NAAC with 'A++' Grade
॥वसुधैव कुटुम्बकम्॥

Name : Atharva Ambilwade
PRN : 25070122055
Class : CSE
Batch : 2025-29

Assignment : 25

Subject: Programming with Java

ASSIGNMENT No : 25

TITLE : Write a Java program using the Thread class to create multiple threads and demonstrate their lifecycle.

Problem Statement : Write a Java program using the Thread class to create multiple threads and demonstrate their lifecycle.

Learning Objectives : To create and manage multiple threads using the Thread class.

Theory :
A thread is the smallest unit of execution within a Java program. Multithreading enables multiple tasks to execute concurrently, improving application performance and responsiveness. Java provides the Thread class for creating threads by extending it and overriding the run() method. A thread passes through different lifecycle stages including New, Runnable, Running, Blocked/Waiting, and Terminated. The scheduler manages thread execution based on system resources. Methods such as start(), sleep(), join(), and interrupt() control thread behavior. Multithreading is commonly used in games, web servers, animation, and background processing applications.

Code :

A25.java > Language Support for Java(TM) by Red Hat > A25

```java
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(getName() + ": " + i);

            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

public class A25 {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();

        t1.start();
        t2.start();
    }
}
```

Output :

```
PS C:\Users\LENOVO\Documents\JAVA Assignments>  &
' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' '
a\jdt_ws\JAVA Assignments_4fd1afc\bin' 'A25'
Thread-0: 1
Thread-0: 2
Thread-1: 2
Thread-0: 3
Thread-1: 3
Thread-1: 4
Thread-0: 4
Thread-0: 5
Thread-1: 5
PS C:\Users\LENOVO\Documents\JAVA Assignments>
```

Conclusion : This program demonstrates multithreading in Java by creating two threads, t1 and t2, using the Thread class. Both threads execute the run() method concurrently and print numbers from 1 to 5, with a delay of 500 milliseconds between each number. The start() method starts each thread, while Thread.sleep() pauses the executing thread temporarily. The output order may vary because thread scheduling is managed by the JVM and operating system.

Exercise

1.Create a program where multiple threads print numbers simultaneously
Ans) Code :

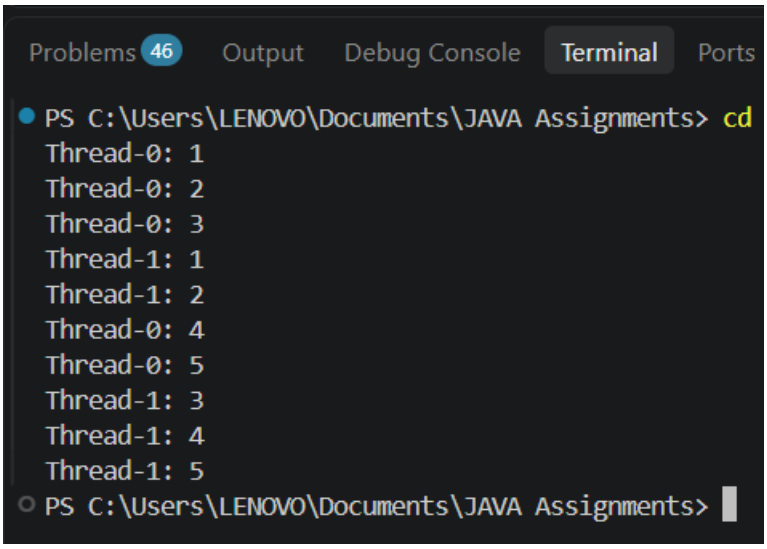
A25E1.java > Java > A25E1

```java
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(getName() + ": " + i);
        }
    }
}

public class A25E1 {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();

        t1.start();
        t2.start();
    }
}
```

Output :


Problems 46
Output
Debug Console
Terminal
Ports
PS C:\Users\LENOVO\Documents\JAVA Assignments> cd
Thread-0: 1
Thread-0: 2
Thread-0: 3
Thread-1: 1
Thread-1: 2
Thread-0: 4
Thread-0: 5
Thread-1: 3
Thread-1: 4
Thread-1: 5
PS C:\Users\LENOVO\Documents\JAVA Assignments>

2. Create a Java program using multiple threads to simulate bank transactions. One thread performs deposit operations, and another performs withdrawal operations, both printing transaction details simultaneously.

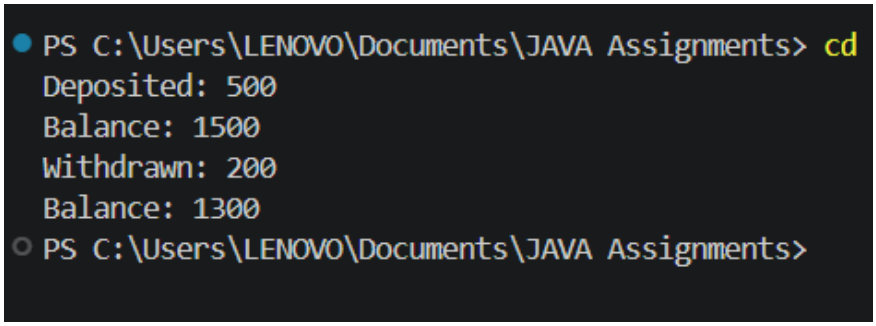
Ans) Code :

A25E2.java > Java > A25E2

```java
class Bank {
    int balance = 1000;
    synchronized void deposit(int amount) {
        balance += amount;
        System.out.println("Deposited: " + amount);
        System.out.println("Balance: " + balance);
    }
    synchronized void withdraw(int amount) {
        balance -= amount;
        System.out.println("Withdrawn: " + amount);
        System.out.println("Balance: " + balance);
    }
}
class Deposit extends Thread {
    Bank b;
    Deposit(Bank b) {
        this.b = b;
    }
    public void run() {
        b.deposit(500);
    }
}
class Withdraw extends Thread {
    Bank b;

    Withdraw(Bank b) {
        this.b = b;
    }
    public void run() {
        b.withdraw(200);
    }
}
public class A25E2 {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        Bank b = new Bank();
        Deposit t1 = new Deposit(b);
        Withdraw t2 = new Withdraw(b);
        t1.start();
        t2.start();
    }
}
```

Output :


PS C:\Users\LENOVO\Documents\JAVA Assignments> cd
Deposited: 500
Balance: 1500
Withdrawn: 200
Balance: 1300
PS C:\Users\LENOVO\Documents\JAVA Assignments>